

C Array Manipulation Translated into MIPS64 Assembly (EduMIPS Simulator)

```
Start here X array5.c X
1 #include<stdio.h>
2
3 void main(){
4
5     char Alpha[30] = {'c','s','c','l','l','2','m','e'}; //STORE Operation in MIPS memory address
6     char Beta[30];
7     char Charlie[30];
8     int x,y=0;
9
10    for (x = 0; x < strlen(Alpha); x++){
11        Beta[x] = Alpha[x]; //STORE Operation in MIPS for another memory address
12        if(x>=4){
13            Charlie[y] = Alpha[x];
14            printf("%c",Charlie[y]); //LOAD Operation in MIPS into a specific register
15        }
16    }
17    printf("\n%c", Beta[0]);
18    printf("\n%s", Beta);
19 }
20
21
```

HINT: There are only eight C lines of code that need to be transformed into either STORE, LOAD, or IMMEDIATE operations in MIPS assembly language using EduMIPS.

Problem Statement:

Translate the given C program that manipulates three-character arrays (**Alpha, Beta, and Charlie**) into equivalent MIPS64 assembly code, runnable in the EduMIPS simulator. Some C lines are already **annotated with comments** indicating the required MIPS operation; your task is to examine the remaining lines and determine where LOAD, STORE, or IMMEDIATE instructions are necessary. While not every C statement has a direct one-to-one translation, all operations that **affect data must be expressed using the appropriate MIPS64 instructions**: memory write operations should be implemented using store instructions such as SB (Store Byte), memory read operations should be implemented using load instructions such as LB or LBU (Load Byte/Load Byte Unsigned), and immediate operations, particularly register assignments, must use DADDIU to transfer values into MIPS registers. Other load or store instructions may also be used when needed for correct data handling, as long as they accurately reflect the intent of the original C code.

What to Expect:

You are expected to produce a runnable MIPS64 assembly code listing in which each relevant C instruction is mapped to its appropriate low-level equivalent. Not every line in the C program has a direct translation, but all operations that **affect data** must be expressed using LOAD, STORE, or IMMEDIATE instructions. Accuracy will be assessed by how well your assembly reflects the **program's flow—initializing array variables and values, iterating over Alpha to copy into Beta, conditionally transferring to Charlie, and printing the outputs**—while ensuring a clear one-to-one correspondence between the essential C logic and its low-level MIPS instructions.

Note: The **final program** must clearly demonstrate only STORE, LOAD, and IMMEDIATE operations derived from the C array manipulations. Once verified in EduMIPS, write your answer neatly on one whole sheet of yellow paper and submit it.